

ДИСЦИПЛИНА	Программирование в операционной системе Линукс
ИНСТИТУТ	Передовая инженерная школа СВЧ-электроники
КАФЕДРА	Передовых технологий
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания по дисциплине
ПРЕПОДАВАТЕЛЬ	Астафьев Рустам Уралович
СЕМЕСТР	1 семестр, 2025/2026 уч. год

Ссылка на материал:

<https://github.com/astafiev-rustam/programming-in-the-linux-operating-system/tree/lecture-1-4>

Лекция №4: Сетевые возможности и отладка

Теоретическая вводная

Сети в Linux: кровеносная система операционной системы

Представьте, что компьютер — это город, а сетевое подключение — это дороги, соединяющие его с другими городами. В мире Linux сетевые возможности встроены глубоко в саму философию системы. В отличие от других операционных систем, где сеть может восприниматься как нечто дополнительное, в Linux она является фундаментальной частью ядра, доступной и настраиваемой на самом низком уровне.

Каждое сетевое взаимодействие начинается с интерфейса — виртуальной или физической двери, через которую компьютер общается с внешним миром. Это может быть Ethernet-порт, Wi-Fi адаптер, или даже виртуальный интерфейс вроде loopback, который позволяет программам общаться внутри одной машины. Linux умеет создавать сложные сетевые конструкции: мосты, туннели, виртуальные частные сети — словно архитектор, строящий развязки и тоннели между городами.

IP-адреса: цифровые паспорта в сети

В мире сетей каждый компьютер нуждается в уникальном идентификаторе — IP-адресе. Это как почтовый адрес в цифровом городе. IPv4 адреса, знакомые нам по формату 192.168.1.1, постепенно уступают место IPv6 с их необъятным пространством адресов. Но адрес — это только половина истории. Маска подсети определяет границы "района", в котором находится компьютер, а шлюз по умолчанию — это главная выездная дорога в другие сети.

DNS-серверы выполняют роль телефонного справочника, преобразуя человеко-читаемые имена like google.com в машинно-читаемые IP-адреса. Когда вы вводите адрес сайта в браузере, ваш компьютер сначала обращается к DNS-серверу с вопросом: "Какой IP-адрес у google.com?" и только потом устанавливает соединение с полученным адресом.

Порты и протоколы: квартиры и языки общения

Если IP-адрес — это адрес дома, то порт — это номер квартиры. В одном здании могут жить разные службы: веб-сервер на 80-м порту, почтовый сервер на 25-м, SSH на 22-м. Всего доступно 65535 портов, от 0 до 1023 — привилегированные порты, зарезервированные для системных служб.

Но чтобы жильцы разных квартир могли понимать друг друга, им нужен общий язык — протокол. TCP напоминает заказное письмо с уведомлением о вручении: он гарантирует доставку данных, проверяет целостность и устанавливает надежное соединение. UDP же похож на обычную открытку: отправил и надеешься, что дойдет. Быстро, но без гарантий.

Сетевые утилиты: инструменты сетевого доктора

Linux предлагает богатейший арсенал инструментов для диагностики сетей. Команда `ip` заменила устаревший `ifconfig` и позволяет не просто просматривать настройки интерфейсов, но и создавать сложные сетевые конфигурации. `ss` показывает, какие программы с какими портами работают, заменяя старичка `netstat`.

`ping` — это простейший способ проверить, жив ли удаленный хост. Он отправляет эхо-запросы и измеряет время ответа. `traceroute` идет дальше — он показывает весь путь пакета от вашего компьютера до цели, выявляя проблемные участки. А когда нужно глубоко изучить, что же именно передается по сети, на помощь приходит `tcpdump` — настоящий рентген для сетевого трафика.

SSH: безопасные ворота для удаленного управления

Secure Shell — это не просто способ удаленного подключения к серверу. Это целая экосистема безопасного управления. SSH использует криптографию, чтобы защитить ваше соединение от посторонних глаз, словно строит зашифрованный туннель между двумя компьютерами.

Но настоящая магия начинается с ключей SSH. Вместо паролей, которые можно подобрать или перехватить, используются пары криптографических ключей: приватный ключ хранится у вас, как отпечаток пальца, а публичный — на сервере, как слепок замка. Когда они совпадают, дверь открывается. Это не только безопаснее, но и удобнее — не нужно постоянно вводить пароли.

Логи и мониторинг: память сети

Сети редко работают идеально всегда. Проблемы возникают то тут, то там. Чтобы их диагностировать, нужно уметь слушать, что система рассказывает о своей сетевой жизни. Логи — это дневник, в котором Linux записывает все важные сетевые события.

Журналы `systemd`, файлы в `/var/log`, специализированные логи демонов — все они хранят ценную информацию. Умение читать эти логи, фильтровать нужное, понимать, что является нормальным поведением, а что — признаком проблемы, это искусство, которое приходит с опытом.

В современном Linux большинство систем используют `systemd`, и `journalctl` становится вашим главным инструментом для просмотра логов. А такие утилиты, как `iftop` и `nethogs`, позволяют в реальном времени наблюдать за сетевой активностью, показывая, какие программы сколько трафика потребляют.

Безопасность: бдительность на страже

Сетевые возможности несут с собой и риски. Открытые порты — это потенциальные входные двери для злоумышленников. Межсетевые экраны вроде `iptables` или `nftables` выполняют роль службы безопасности, решая, какие пакеты пропускать, а какие блокировать.

Понимание основ сетевой безопасности становится критически важным. Простые практики — использование нестандартных портов для служб, отключение ненужных служб, регулярное обновление

системы — могут значительно повысить вашу защищенность.

Практические примеры

Пример 1: Основы работы с утилитой `ip`

Цель: Научиться использовать современную утилиту `ip` для управления сетевыми интерфейсами.

```
# 1. Просмотрим все сетевые интерфейсы
ip addr show
# или сокращенно:
ip a

# 2. Посмотрим только активные интерфейсы
ip link show up

# 3. Просмотрим таблицу маршрутизации
ip route show
# Это покажет, через какие шлюзы пакеты идут в разные сети

# 4. Посмотрим статистику по сетевым интерфейсам
ip -s link
# Покажет количество переданных/принятых пакетов и ошибки

# 5. Временно отключим и включим интерфейс (замените eth0 на ваш интерфейс)
sudo ip link set eth0 down
sudo ip link set eth0 up

# 6. Добавим временный IP-адрес интерфейсу
sudo ip addr add 192.168.1.100/24 dev eth0

# 7. Удалим временный IP-адрес
sudo ip addr del 192.168.1.100/24 dev eth0

# 8. Просмотрим ARP-таблицу (соответствие IP и MAC-адресов в локальной сети)
ip neigh show
```

Пример 2: Диагностика подключения с помощью `ping`

Цель: Освоить использование `ping` для базовой диагностики сети.

```
# 1. Простейшая проверка доступности хоста
ping google.com

# 2. Ограничим количество пакетов (удобно для автоматических проверок)
ping -c 4 google.com

# 3. Установим интервал между пакетами (по умолчанию 1 секунда)
ping -i 2 -c 3 google.com
```

```
# 4. Проверим доступность локального интерфейса (loopback)
ping -c 2 127.0.0.1

# 5. Используем ping для проверки DNS-работоспособности
ping -c 2 8.8.8.8
# Если пинг по IP работает, а по имени нет - проблема в DNS

# 6. Установим размер пакета (полезно для диагностики проблем с MTU)
ping -s 1400 -c 2 google.com

# 7. Проверим доступность порта с помощью ping (не стандартно, но иногда работает)
# Обычно ping работает на уровне ICMP, не TCP/UDP
ping -c 1 google.com

# 8. Непрерывный ping для мониторинга стабильности соединения
# ping google.com
# Прервать - Ctrl+C
```

Пример 3: Анализ сетевых соединений с помощью ss

Цель: Научиться использовать **ss** для просмотра сетевых соединений.

```
# 1. Просмотрим все установленные соединения
ss -tuna

# Расшифровка ключей:
# -t - TCP соединения
# -u - UDP соединения
# -n - показывать цифровые адреса (не резолвить имена)
# -a - все соединения (включая слушающие порты)

# 2. Посмотрим только слушающие порты (какие службы ожидают подключений)
ss -tunlp

# 3. Посмотрим соединения в конкретном состоянии (например, ESTABLISHED)
ss -tun state established

# 4. Найдём, какой процесс использует порт 22 (SSH)
ss -tunlp sport = :22

# 5. Посмотрим статистику по типам соединений
ss -s

# 6. Мониторим соединения в реальном времени
watch -n 1 'ss -tun state established'

# 7. Сравним с устаревшей утилитой netstat (если установлена)
netstat -tunlp
```

Пример 4: Трассировка маршрута с помощью traceroute

Цель: Научиться определять путь пакетов до целевого хоста.

```
# 1. Установим утилиту traceroute если не установлена
sudo apt install traceroute

# 2. Выполним трассировку до удаленного хоста
traceroute google.com

# 3. Используем альтернативную утилиту tracpath
tracpath google.com

# 4. Установим максимальное количество хопов
traceroute -m 10 google.com

# 5. Используем определенный источник для трассировки (если несколько интерфейсов)
traceroute -i eth0 google.com

# 6. Трассировка с использованием UDP (по умолчанию) vs ICMP
traceroute -I google.com # ICMP
traceroute -T google.com # TCP

# 7. Сохраним результат трассировки в файл для анализа
traceroute google.com > traceroute_result.txt
cat traceroute_result.txt
```

Пример 5: Сканирование сети с помощью nmap

Цель: Освоить базовые возможности nmap для анализа сети.

```
# 1. Установим nmap
sudo apt install nmap

# 2. Просканируем отдельный хост
nmap localhost

# 3. Просканируем диапазон портов на конкретном хосте
nmap -p 1-1000 192.168.1.1

# 4. Определим версии служб на открытых портах
nmap -sV 192.168.1.1

# 5. Просканируем подсеть на наличие активных хостов
nmap -sn 192.168.1.0/24

# 6. Быстрое сканирование (только основные порты)
nmap -F 192.168.1.1

# 7. Агрессивное сканирование (больше информации, но заметнее)
```

```
nmap -A 192.168.1.1

# 8. Сохраним результат в файл
nmap -oN scan_result.txt 192.168.1.1
```

Пример 6: Анализ сетевого трафика с помощью tcpdump

Цель: Научиться использовать tcpdump для глубокого анализа сетевого трафика.

```
# 1. Установим tcpdump
sudo apt install tcpdump

# 2. Прослушаем весь трафик на интерфейсе (осторожно: много вывода!)
sudo tcpdump -i eth0

# 3. Ограничим количество пакетов
sudo tcpdump -i eth0 -c 10

# 4. Будем выводить IP вместо имен хостов
sudo tcpdump -i eth0 -n -c 5

# 5. Перехватим только HTTP трафик (порт 80)
sudo tcpdump -i eth0 -n 'tcp port 80'

# 6. Перехватим трафик к/от конкретного хоста
sudo tcpdump -i eth0 host google.com

# 7. Сохраним трафик в файл для последующего анализа
sudo tcpdump -i eth0 -w capture.pcap -c 20

# 8. Прочитаем сохраненный файл
tcpdump -r capture.pcap

# 9. Более сложный фильтр: только SYN-пакеты (начало TCP-соединений)
sudo tcpdump -i eth0 'tcp[tcpflags] == tcp-syn'
```

Пример 7: Настройка SSH и работа с ключами

Цель: Научиться настраивать SSH и работать с ключами для безопасного доступа.

```
# 1. Проверим, запущен ли SSH сервер
sudo systemctl status ssh

# 2. Установим SSH сервер если не установлен
sudo apt install openssh-server

# 3. Сгенерируем SSH ключевую пару (если еще нет)
ssh-keygen -t rsa -b 4096 -C "your_email@example.com"
# Следуйте инструкциям, можно оставить парольную фразу пустой для теста
```

```
# 4. Скопируем публичный ключ на удаленный сервер
ssh-copy-id username@remote_server

# 5. Подключимся к удаленному серверу с использованием ключа
ssh username@remote_server

# 6. Выполним команду на удаленном сервере без интерактивного входа
ssh username@remote_server 'ls -la'

# 7. Создадим SSH туннель для безопасного доступа к службе
ssh -L 8080:localhost:80 username@remote_server
# Теперь localhost:8080 на вашей машине будет перенаправлен на порт 80 удаленного сервера

# 8. Настроим файл конфигурации SSH для удобства
mkdir -p ~/.ssh
cat >> ~/.ssh/config << 'EOF'
Host myserver
    HostName remote_server.com
    User username
    Port 22
    IdentityFile ~/.ssh/id_rsa
EOF
# Теперь можно подключаться просто: ssh myserver
```

Пример 8: Работа с сетевыми службами и портами

Цель: Научиться управлять сетевыми службами и анализировать использование портов.

```
# 1. Установим утилиту lsof для анализа открытых файлов (включая сетевые соединения)
sudo apt install lsof

# 2. Посмотрим, какие процессы используют сетевые порты
sudo lsof -i

# 3. Найдем процесс, использующий конкретный порт
sudo lsof -i :80

# 4. Посмотрим открытые порты для IPv4 и IPv6
sudo lsof -i -P -n

# 5. Установим и настроим простой веб-сервер для демонстрации
sudo apt install nginx
sudo systemctl start nginx

# 6. Проверим, что nginx слушает порт 80
sudo ss -tunlp | grep nginx

# 7. Протестируем подключение к веб-серверу
```

```
curl -I http://localhost

# 8. Остановим веб-сервер и проверим освобождение порта
sudo systemctl stop nginx
sudo ss -tunlp | grep :80
```

Пример 9: Мониторинг сетевого трафика в реальном времени

Цель: Научиться использовать инструменты для мониторинга сетевой активности.

```
# 1. Установим инструменты для мониторинга
sudo apt install nethogs iftop bmon

# 2. Запустим nethogs для просмотра трафика по процессам
sudo nethogs eth0
# Покажет, какие программы сколько трафика используют

# 3. Запустим iftop для просмотра трафика по хостам
sudo iftop -i eth0
# Покажет трафик между вашей машиной и другими хостами

# 4. Используем bmon для красивого мониторинга интерфейсов
bmon

# 5. Мониторинг с помощью /proc файловой системы
cat /proc/net/dev
# Покажет статистику по всем сетевым интерфейсам

# 6. Создадим простой скрипт для мониторинга
while true; do
    clear
    date
    echo "=== Сетевые интерфейсы ==="
    ip -s link show eth0
    echo "=== Активные соединения ==="
    ss -tun state established | head -10
    sleep 2
done
```

Пример 10: Настройка статического IP и разрешения имен

Цель: Научиться настраивать статическую сетевую конфигурацию.

```
# 1. Посмотрим текущую конфигурацию сети
cat /etc/netplan/*.yaml

# 2. Создадим backup текущей конфигурации
sudo cp /etc/netplan/01-netcfg.yaml /etc/netplan/01-netcfg.yaml.backup
```



```
# 3. Отредактируем конфигурацию netplan (замените на ваш конфиг файл)
sudo nano /etc/netplan/01-netcfg.yaml

# Пример конфигурации для статического IP:
"""
network:
  version: 2
  renderer: networkd
  ethernets:
    eth0:
      addresses:
        - 192.168.1.100/24
      gateway4: 192.168.1.1
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
"""

# 4. Применим новую конфигурацию
sudo netplan apply

# 5. Проверим, что настройки применились
ip addr show eth0
ip route show

# 6. Проверим работу DNS
nslookup google.com
dig google.com

# 7. Посмотрим файл resolv.conf
cat /etc/resolv.conf
```

Пример 11: Диагностика DNS проблем

Цель: Научиться диагностировать и решать проблемы с DNS.

```
# 1. Проверим базовое разрешение имен
nslookup google.com

# 2. Используем dig для более детальной информации
dig google.com

# 3. Проследуем всю цепочку DNS запросов
dig +trace google.com

# 4. Проверим конкретный DNS сервер
dig @8.8.8.8 google.com

# 5. Посмотрим, какие DNS серверы использует система
cat /etc/resolv.conf

# 6. Проверим кеш DNS (если используется systemd-resolved)
```

```
sudo systemd-resolve --status

# 7. Почистим DNS кеш
sudo systemd-resolve --flush-caches

# 8. Проверим связность с DNS сервером
ping -c 2 8.8.8.8

# 9. Протестируем разные DNS серверы
for dns in 8.8.8.8 1.1.1.1 9.9.9.9; do
    echo "Testing $dns:"
    dig @$dns google.com | grep "Query time"
done
```

Пример 12: Работа с сетевыми файрволами (ufw)

Цель: Научиться базовому управлению сетевым файрволом.

```
# 1. Установим ufw (Uncomplicated Firewall)
sudo apt install ufw

# 2. Проверим статус
sudo ufw status

# 3. Разрешим SSH подключения (очень важно сделать ДО включения!)
sudo ufw allow ssh
# или по порту
sudo ufw allow 22/tcp

# 4. Разрешим HTTP и HTTPS трафик
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp

# 5. Включим файрвол
sudo ufw enable

# 6. Проверим правила
sudo ufw status numbered

# 7. Запретим трафик с конкретного IP
sudo ufw deny from 192.168.1.50

# 8. Разрешим трафик только для конкретной подсети
sudo ufw allow from 192.168.1.0/24

# 9. Удалим правило по номеру
sudo ufw delete 2

# 10. Отключим файрвол (если нужно)
sudo ufw disable
```

Пример 13: Анализ сетевых проблем с помощью системных логов

Цель: Научиться использовать логи для диагностики сетевых проблем.

```
# 1. Просмотрим системные логи, связанные с сетью
sudo journalctl -u systemd-networkd

# 2. Посмотрим логи за последний час
sudo journalctl -u systemd-networkd --since "1 hour ago"

# 3. Будем следить за логами в реальном времени
sudo journalctl -u systemd-networkd -f

# 4. Посмотрим логи демона SSH
sudo journalctl -u ssh

# 5. Ищем ошибки в системных логах
sudo journalctl -p err

# 6. Просмотрим логи из традиционных файлов в /var/log
sudo tail -f /var/log/syslog

# 7. Создадим свой демон для логирования сетевых событий
cat > network_monitor.sh << 'EOF'
#!/bin/bash
while true; do
    TIMESTAMP=$(date '+%Y-%m-%d %H:%M:%S')
    CONNECTIONS=$(ss -tun state established | wc -l)
    echo "[${TIMESTAMP}] Established connections: $((CONNECTIONS - 1))" >>
/var/log/network_monitor.log
    sleep 30
done
EOF

chmod +x network_monitor.sh
# Запуск: sudo ./network_monitor.sh &
```

Пример 14: Создание сетевых туннелей и VPN

Цель: Освоить основы создания сетевых туннелей.

```
# 1. Создадим простой TCP туннель с помощью netcat
# На принимающей стороне:
nc -l -p 8888

# На отправляющей стороне:
echo "Hello through tunnel" | nc localhost 8888

# 2. Создадим SSH туннель для доступа к удаленной базе данных
# ssh -L 5432:localhost:5432 user@remote_server
```

```
# Теперь подключение к localhost:5432 будет перенаправлено на remote_server:5432

# 3. Установим и настроим OpenVPN (упрощенный пример)
sudo apt install openvpn

# 4. Создадим простой VPN с помощью ssh
# Создадим туннель для всего трафика через удаленный сервер
# ssh -D 1080 -N user@remote_server
# Затем настраиваем браузер использовать SOCKS прокси localhost:1080

# 5. Проверим туннельные интерфейсы
ip tun show

# 6. Создадим простой GRE туннель между двумя хостами
# На первом хосте:
sudo ip tunnel add gre1 mode gre remote 192.168.1.2 local 192.168.1.1
sudo ip addr add 10.0.0.1/24 dev gre1
sudo ip link set gre1 up

# На втором хосте:
sudo ip tunnel add gre1 mode gre remote 192.168.1.1 local 192.168.1.2
sudo ip addr add 10.0.0.2/24 dev gre1
sudo ip link set gre1 up

# 7. Проверим туннель
ping 10.0.0.2
```

Пример 15: Продвинутая диагностика и бенчмаркинг сети

Цель: Освоить продвинутые техники диагностики производительности сети.

```
# 1. Установим инструменты для тестирования производительности
sudo apt install iperf3 speedtest-cli

# 2. Запустим iperf3 сервер на одной машине
iperf3 -s

# 3. На другой машине протестируем скорость до сервера
iperf3 -c server_ip

# 4. Протестируем скорость загрузки
iperf3 -c server_ip -R

# 5. Используем speedtest-cli для тестирования интернета
speedtest-cli

# 6. Создадим скрипт для мониторинга качества сети
cat > network_quality.sh << 'EOF'
#!/bin/bash
SERVER="8.8.8.8"
LOG_FILE="/var/log/network_quality.log"
```

```
while true; do
    TIMESTAMP=$(date '+%Y-%m-%d %H:%M:%S')

    # Проверка пинга
    PING_RESULT=$(ping -c 3 $SERVER 2>/dev/null | tail -1)
    PING_TIME=$(echo $PING_RESULT | awk -F '/' '{print $5}')

    # Проверка потери пакетов
    PACKET_LOSS=$(echo $PING_RESULT | grep -oP '\d+(?=%)' || echo "100")

    echo "[${TIMESTAMP}] Ping: ${PING_TIME}ms, Loss: ${PACKET_LOSS}%" >> $LOG_FILE
    sleep 60
done
EOF

chmod +x network_quality.sh

# 7. Проверим MTU (Maximum Transmission Unit) пути
ping -M do -s 1472 -c 2 google.com
# Если пинг не проходит, уменьшаем размер пакета пока не заработает

# 8. Мониторинг использования полосы пропускания
sudo iftop -i eth0 -B
```